

Models of Language and Conversation

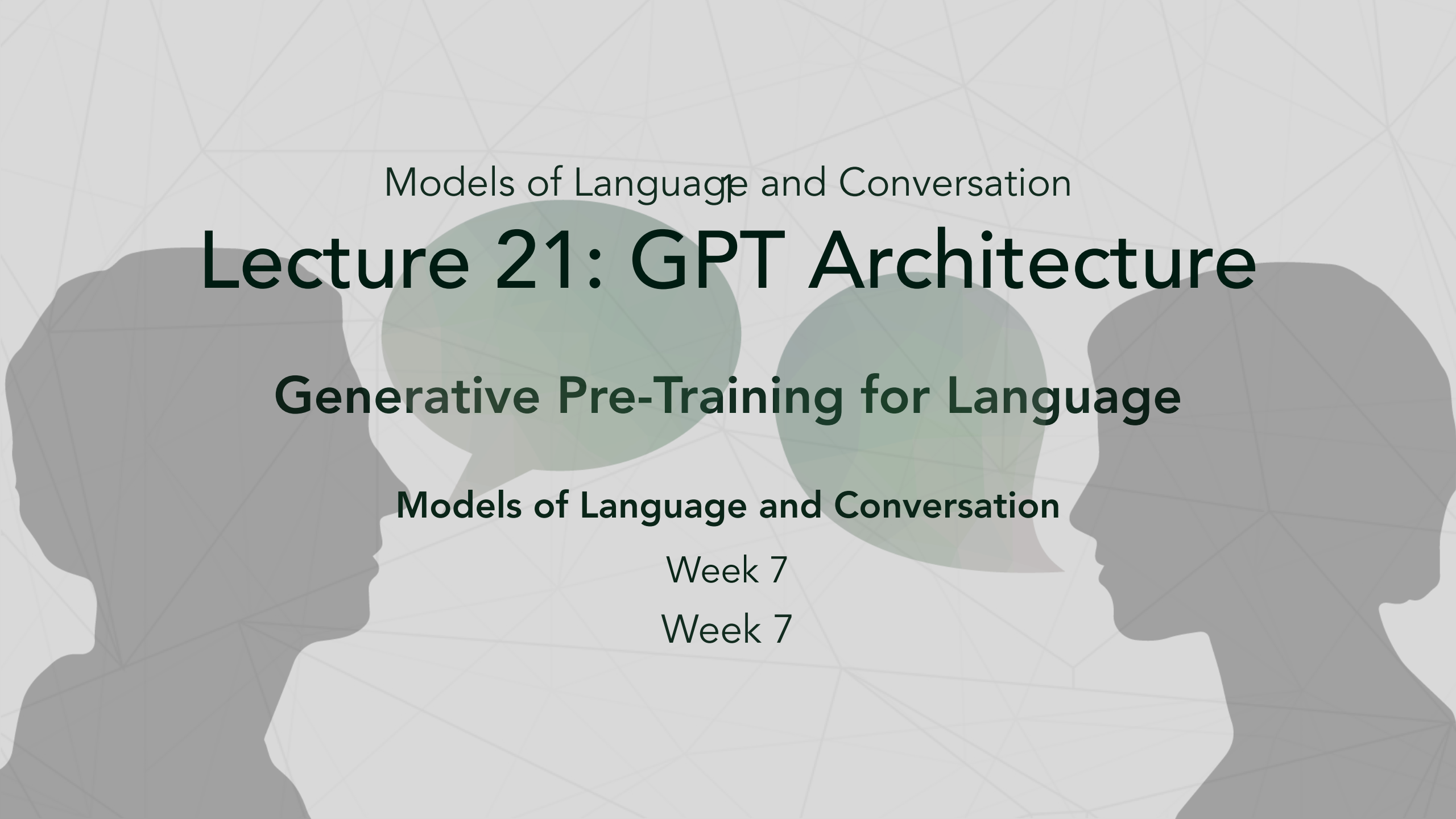
Lecture 21: GPT Architecture

Generative Pre-Training for Language

Models of Language and Conversation

Week 7

Week 7



Today's Journey

1. **The GPT Revolution** - Pre-training + fine-tuning paradigm
2. **Transformer Decoder** - Architecture walkthrough
3. **Masked Self-Attention** - The core innovation
4. **Two-Stage Training** - Pre-training and fine-tuning
5. **Hands-on Code** - Building GPT blocks in PyTorch
6. **Key Results** - What made GPT successful

The GPT Revolution

Discussion

What made GPT different from previous language models?

Key Innovation: Pre-training + Fine-tuning

- **Pre-train** on massive unlabeled text (unsupervised)
- **Fine-tune** on specific tasks (supervised)
- Transfer learning for NLP
- One model, many tasks

Paradigm Shift

Before GPT: Task specific models trained from scratch

From BERT to GPT: Different Approaches

BERT (2018):

- **Masked Language Model**
- Bidirectional context
- Fill in the blank
- Great for understanding
- Not designed for generation

GPT (2018):

- **Autoregressive Model**
- Left-to-right (causal)
- Predict next token
- Natural for generation
- Can also understand

The Transformer Decoder

GPT uses the Transformer *decoder* architecture

Data Flow:

1. Input tokens
2. Token + Position embeddings
3. N x Transformer blocks:

Masked Self-attention

Key Components:

- **Embeddings:** Map tokens to vectors
- **Masked Attention:** Only see past tokens
- **Feed-Forward:** Process each position

Transformer Decoder: Worked Example

Example: Processing "The cat sat"

```
1 Step 1: Token Embeddings
2 "The" → [0.2, -0.1, 0.8, ...] (768-dim vector)
3 "cat" → [0.5, 0.3, -0.2, ...]
4 "sat" → [-0.1, 0.7, 0.4, ...]
5
6 Step 2: Add Position Embeddings
7 Position 0 embedding + "The" embedding
8 Position 1 embedding + "cat" embedding
9 Position 2 embedding + "sat" embedding
```


Masked Self-Attention

The Core Innovation: Causal Masking

Attention Matrix for "The cat sat on":

	The	cat	sat	on
The	1.0	-inf	-inf	-inf
cat	0.3	0.7	-inf	-inf
sat	0.2	0.3	0.5	-inf
on	0.1	0.2	0.4	1.0

Causal Mask: Code Implementation

```
1 import torch
2
3 def create_causal_mask(seq_len):
4     """Create lower-triangular mask for causal attention."""
5     # Create a matrix of ones
6     mask = torch.ones(seq_len, seq_len)
7     # Keep only lower triangle (including diagonal)
8     mask = torch.tril(mask)
```

This mask is applied before softmax to prevent attending to future tokens!

Position Encoding in GPT

Why we need position information:

The Problem

Self-attention is *permutation invariant* - it doesn't inherently know word order!

GPT's Solution: Learned Position Embeddings

- Each position gets a learnable embedding
- Added to token embeddings:

```
embedding = token_emb + pos_emb
```

- Model learns position patterns during training
- Maximum sequence length: 512 tokens (GPT-1)

Position Embedding: Code Example

```
1 import torch
2 import torch.nn as nn
3
4 class GPTEmbedding(nn.Module):
5     def __init__(self, vocab_size, d_model, max_seq_len):
6         super().__init__()
7         # Token embeddings: word → vector
8         self.token_embed = nn.Embedding(vocab_size, d_model)
9         # Position embeddings: position → vector
10        self.pos_embed = nn.Embedding(max_seq_len, d_model)
11
12        def forward(self, x):
13            seq_len = x.size(1)
14            # Create position indices [0, 1, 2, ..., seq_len-1]
15            positions = torch.arange(seq_len, device=x.device)
16            # Combine: token embedding + position embedding
17            return self.token_embed(x) + self.pos_embed(positions)
```

10

Week 7

continued...

Position Embedding: Code Example

```
18  
19 # Example usage  
20 embed = GPTEmbedding(vocab_size=50000, d_model=768, max_seq_len=512)  
21 tokens = torch.tensor([[101, 2054, 2003]]) # "The cat sat"  
22 embeddings = embed(tokens) # Shape: (1, 3, 768)
```

Week 7

Models of Language and Conversation

GPT-1 Model Specifications

Component	Value
Layers	12
Hidden size	768
Attention heads	12
Max sequence length	512 tokens
Vocabulary size	40,000 (BPE)

GPT Architecture in Code

```
1 import torch.nn as nn
2
3 class GPTBlock(nn.Module):
4     def __init__(self, d_model, n_heads):
5         super().__init__()
6         self.attention = MaskedMultiHeadAttention(d_model, n_heads)
7         self.norm1 = nn.LayerNorm(d_model)
8         self.ffn = nn.Sequential(
9             nn.Linear(d_model, 4 * d_model),
10            nn.GELU(), # GPT uses GELU, not ReLU
11            nn.Linear(4 * d_model, d_model)
12        )
13        self.norm2 = nn.LayerNorm(d_model)
14
15    def forward(self, x):
16        # Pre-norm architecture (LayerNorm before sublayer)
17        # Self-attention with residual connection
18        x = x + self.attention(self.norm1(x))
19        # Feed-forward with residual connection
```

13

Week 7

continued...

GPT Architecture in Code

```
20 | x = x + self.ffn(self.norm2(x))  
21 | return x
```

Week 7

Complete GPT Model Structure

```
1 class GPT(nn.Module):
2     def __init__(self, vocab_size, d_model, n_layers, n_heads, max_len):
3         super().__init__()
4         self.token_embed = nn.Embedding(vocab_size, d_model)
5         self.pos_embed = nn.Embedding(max_len, d_model)
6         self.blocks = nn.ModuleList([
7             GPTBlock(d_model, n_heads) for _ in range(n_layers)
8         ])
9         self.norm = nn.LayerNorm(d_model)
10        self.head = nn.Linear(d_model, vocab_size)
11
12        def forward(self, x):
13            seq_len = x.size(1)
14            positions = torch.arange(seq_len, device=x.device)
15            x = self.token_embed(x) + self.pos_embed(positions)
16            for block in self.blocks:
17                x = block(x)
18            x = self.norm(x)
```

15

Week 7

continued...

Complete GPT Model Structure

```
19 | logits = self.head(x) # (batch, seq_len, vocab_size)  
20 | return logits
```

Week 7

Two-Stage Training

Stage 1: Pre-training

- Dataset: BooksCorpus (5B tokens)
- Objective: Next token prediction
- Duration: Weeks of training
- Result: General language

Stage 2: Fine-tuning

- Dataset: Task-specific (1K-100K examples)
- Objective: Task loss + LM loss
- Duration: Hours of training¹⁷
- Result: Task-specialized

Pre-training: Language Modeling

Objective: Predict next token

Example Training Sequence

Text: "The quick brown fox jumps over the lazy dog"

Training examples (teacher forcing):

- Input: "The" -> Target: "quick"
- Input: "The quick" -> Target: "brown"
- Input: "The quick brown" -> Target: "fox"
- Input: "The quick brown fox" -> Target: "jumps"
- ...and so on

Self-supervised learning:

Pre-training: Worked Example

How loss is computed for one sequence:

```
1 # Input sequence: "The cat sat on the"
2 tokens = [101, 2054, 2003, 2006, 1996] # Token IDs
3
4 # Model predicts probability distribution for each position
5 # Position 0: P(next | "The") = {"cat": 0.3, "dog": 0.2, ...}
6 # Position 1: P(next | "The cat") = {"sat": 0.4, "ran": 0.1, ...}
7 # etc.
8
9 # Target tokens (shifted by 1)
10 targets = [2054, 2003, 2006, 1996, 2221] # "cat sat on the mat"
```

Fine-tuning for Downstream Tasks

Task Format: Input + Delimiter + Label

Text Classification:

1	[START] This movie is amazing! [DELIM]
---	---

Model predicts: "Positive"

Question Answering:

1	[START] Context [DELIM] Question [DELIM]
---	--

Model predicts:

Fine-tuning: Concrete Example

Sentiment Classification Example:

```
1 # Training example
2 text = "This movie was absolutely fantastic!"
3 label = "positive"
4
5 # Format for GPT
6 input_text = "[START] This movie was absolutely fantastic! [DELIM]"
7 # Tokenize: [50256, 1212, 3807, 373, 5765, 12779, 0, 50257]
8
9 # During fine-tuning:
10 # 1. GPT processes the input
11 # 2. Take the hidden state at [DELIM] position
12 # 3. Pass through classification head
13 logits = classification_head(hidden_state) # [pos_score, neg_score]
14 # 4. Compute cross-entropy with label
15 loss = cross_entropy(logits, label_id)
16
17 # Fine-tuning hyperparameters
18 learning_rate = 6.25e-5 # Much lower than pre-training!
```

21

Week 7

continued...

Fine-tuning: Concrete Example

```
19 | batch_size = 32  
20 | epochs = 3
```

Week 7

2. Add classification head (optional)

- Linear layer for classification
- Or use language modeling head

3. Train with supervised objective

- Cross-entropy loss
- Much lower learning rate
- Few epochs (3-5)

GPT-1 Results

Performance on GLUE Benchmark:

Task	Previous SOTA	GPT-1
Question Answering	86.7	88.1
Semantic Similarity	85.0	85.8
Text	83.0	84.0

Zero-Shot and Few-Shot Learning

Discussion

What if we could use the model *without* fine-tuning?

Definitions:

- **Zero-shot:** No task-specific training examples
- **Few-shot:** A few examples (1-10) as context
- **Fine-tuning:** Full supervised training

GPT-1 observations:

- Fine-tuning works best

Zero-Shot vs Fine-tuning: Example

Task: Sentiment Classification

Zero-shot (GPT-1):

1	Input:
	"Review:
	Great movie!
2	Sentiment:"
3	Output: ???
	(unreliable)

After Fine-tuning:

1	Input: "
	[START]
	Great movie!
	[DELIM]"
2	Output:
	"Positive"

26

Visualizing What GPT Learns

Layer-by-layer analysis of GPT representations:

Layer	What it learns	Example
1-3	Word order, grammar	"The cat" vs "cat The"
4-6	Word meanings, syntax	Subject-verb agreement
7-9	Semantic	"bank" in

The Generative Pre-training Paradigm

Why "Generative Pre-Training" was revolutionary:

1. Unified Architecture

- Same model for all tasks
- vs. task-specific architectures

2. Transfer Learning

Limitations of GPT-1

What GPT-1 couldn't do well:

- **Zero-shot performance:** Needed fine-tuning for each task
- **Model size:** 117M parameters (small by today's standards)
- **Training data:** Limited to ~5B tokens
- **Context length:** Only 512 tokens
- **Generation quality:** Sometimes incoherent
- **Factual accuracy:** Prone to hallucinations

Comparison with BERT

Aspect	BERT	GPT
Training objective	Masked LM	Autoregressive LM
Context	Bidirectional	Left-to-right
Best for	Understanding	Generation
Fine-tuning	Task-specific heads	Unified format

- Unsupervised pre-training on massive text
- Supervised fine-tuning on task data

3. Masked self-attention is key

- Enables autoregressive generation
- Prevents information leakage

4. Transfer learning for NLP

Readings

Required Reading

Radford et al. (2018) - "Improving Language Understanding by Generative Pre-Training"

[[PDF](#)]

Recommended Readings

- **Vaswani et al. (2017)** - "Attention is All You Need" [[ArXiv](#)]
- **Devlin et al. (2018)** - "BERT: Pre-training of Deep Bidirectional Transformers" [[ArXiv](#)]
- **The Illustrated GPT-2** by Jay Alammar [[Blog](#)]

Next Lecture Preview

Lecture 22: Scaling Up to GPT-3 and Beyond

- GPT-2: Language models as unsupervised multitask learners
- GPT-3: Few-shot learning at scale
- Scaling laws: Why bigger is better
- Emergent abilities of large language models
- From GPT-3 to ChatGPT

Questions?

Week 7